

提示词工程

提示工程（Prompt Engineering）是一个新兴学科，关注提示词的开发和优化，帮助用户将大语言模型（LLM）用于各场景和研究领域。掌握了提示工程相关技能将有助于用户更好的了解大型语言模型的能力和局限性。研究人员可利用提示工程来提高大语言模型处理复杂任务场景的能力，如问答和算术推理能力；开发人员可通过提示工程设计和研发出强大的技术，实现和大语言模型或其他生态工具的高效接轨。

模型参数

使用提示词时，通常会通过 API 或直接与大语言模型进行交互。你可以通过配置一些参数以获得不同的提示结果，在上一个章节中我们有提到过。调整这些设置对于提高响应的可靠性非常重要，您可能需要进行一些实验才能找出适合您的用例的正确设置，如下所示：

- Temperature**：简单来说，`temperature` 的参数值越小，模型就会返回越确定的一个结果。如果调高该数值，大语言模型可能会返回更随机的结果，也就是说这可能会带来更多样化或更具创造性的产出。在实际应用方面，对于质量保障（QA）等任务，我们可以设置更低的 `temperature` 值，以促使模型基于事实返回更真实和简洁的结果。对于诗歌生成或其他创造性任务，适度地调高 `temperature` 参数值可能会更好。
- Top P**：使用 `top_p` 也可以用来控制模型返回结果的确定性。如果你需要准确和事实的答案，就把参数值调低。如果你在寻找更多样化的响应，可以将其值调高点。使用Top-P意味着只有词元集合（tokens）中包含 `top_p` 概率质量的才会被考虑用于响应，因此较低的 `top_p` 值会选择最有信心的响应。这意味着较高的 `top_p` 值将使模型考虑更多可能的词语，包括不太可能的词语，从而导致更多样化的输出。一般建议是改变 Temperature 和 Top-P 其中一个参数就行，不用两个都调整。
- Max Tokens**：您可以通过调整 `max_tokens` 来控制大模型生成的 token 数。指定 Max Tokens 有助于防止大模型生成冗长或不相关的响应并控制成本。
- Stop Sequence**：模型会在生成过程中实时检查输出内容，如果遇到跟 `stop` 参数匹配的序列，就会中止生成，即使 `max_tokens` 没用完。指定 `stop` 参数是控制大模型响应长度和结构的另一种方法。
- Frequency Penalty**：`frequency_penalty` 是对下一个生成的 token 进行惩罚，这个惩罚和 token 在响应和提示中已出现的次数成比例，`frequency_penalty` 越高，某个词再次出现的可能性就越小，这个设置通过给 重复数量多的 Token 设置更高的惩罚来减少响应中单词的重复。
- Presence Penalty**：`presence_penalty` 也是对重复的 token 施加惩罚，但与 `frequency_penalty` 不同的是，惩罚对于所有重复 token 都是相同的。出现两次的 token 和出现 10 次的 token 会受到相同的惩罚。此设置可防止模型在响应中过于频繁地生成重复的词。如果您希望模型生成多样化或创造性的文本，您可以设置更高的 `presence_penalty`，如果您希望模型生成更专注的内容，您可以设置更低的 `presence_penalty`。与 `temperature` 和 `top_p` 一样，一般建议是改变 `frequency_penalty` 和 `presence_penalty` 其中一个参数就行，不要同时调整两个。

如果不清楚上面的参数如何设置比较好，可以参考下面提供的设置建议：

- 精确性任务**（问答、代码生成）：

```
temperature: 0.0–0.3
top_p: 0.1–0.3
max_length: 根据需求调整，例如 100–300
stop: 根据上下文设置
frequency_penalty: 0.0–0.5
presence_penalty: 0.0–0.5
```

2. 创造性任务（写作、故事生成）：

```
temperature: 0.7–1.0
top_p: 0.8–1.0
max_length: 根据需求调整, 例如 300–1000
stop: 根据上下文设置
frequency_penalty: 0.5–1.5
presence_penalty: 0.5–2.0
```

3. 探索性任务（高随机性输出）：

```
temperature: 1.2–2.0
top_p: 0.9–1.0
max_length: 根据需求调整, 例如 200–500
stop: 根据上下文设置
frequency_penalty: 0.5–1.0
presence_penalty: 1.0–2.0
```

提示词的格式

标准提示词应该遵循以下格式：

<问题>？

或

<指令>

你可以将其格式化为问答（QA）格式，这在许多问答数据集中是标准格式，如下所示：

Q: <问题>？
A:

当像上面那样提示时，这也被称为**零样本提示**（Zero-Shot Prompt），即你直接提示模型给出一个回答，而没有任何关于你希望它完成的任务的示例或示范。一些大型语言模型具备进行零样本提示的能力，但这取决于手头任务的复杂性和知识，以及模型被训练以在其上表现良好的任务。

具体的零样本提示示例如下：

提示：

Q: What is prompt engineering?

对于一些较新的模型，你可以省略“Q:”部分，因为模型会根据序列（译注：输入的提示词）的构成将其理解为问答任务。换句话说，提示可以简化如下：

提示词

What is prompt engineering?

基于以上标准格式（format），一种流行且有效的提示技术被称为**少样本提示**（Few-Shot Prompt），其中你提供示例（即示范）。你可以按照以下格式组织少样本提示：

```
<问题>?
<答案>
<问题>?
<答案>
<问题>?
<答案>
<问题>?
```

问答格式的版本看起来像这样：

```
Q: <问题>?
A: <答案>
Q: <问题>?
A: <答案>
Q: <问题>?
A: <答案>
Q: <问题>?
A:
```

请记住，使用问答格式并非必须。提示格式取决于手头的任务。例如，你可以执行一个简单的分类任务，并给出如下所示的示例来给任务示范：

提示词：

```
This is awesome! // Positive
This is bad! // Negative
Wow that movie was rad! // Positive
What a horrible show! //
```

输出：

Negative

语言模型可以基于一些说明了解和学习某些任务，而小样本提示正好可以赋能上下文学习能力。

提示词要素

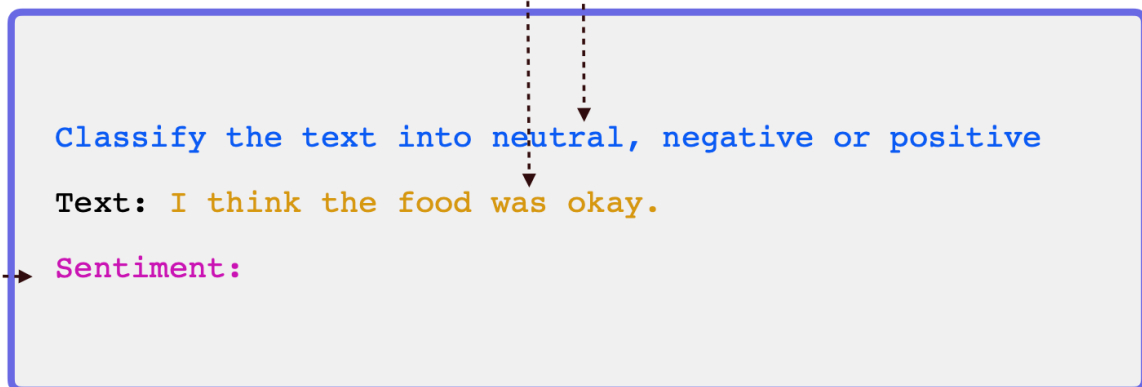
提示词是由一些要素组成的，如下图所示，它包括以下任意要素：

1. **指令**：想要模型执行的特定任务或指令。
2. **上下文**：包含外部信息或额外的上下文信息，引导语言模型更好地响应。
3. **输入数据**：用户输入的内容或问题。

4. 输出指示：指定输出的类型或格式。

- A prompt is composed with the following components:

- Instructions
- Context
- Input data
- Output indicator



下面是一个简单的提示，旨在完成文本分类任务：

提示词

```
请将文本分为中性、否定或肯定
文本：我觉得食物还可以。
情绪：
```

在上面的提示示例中，指令是“将文本分类为中性、否定或肯定”。输入数据是“我认为食物还可以”部分，使用的输出指示是“情绪：”。请注意，此基本示例不使用上下文，但也可以作为提示的一部分提供。例如，此文本分类提示的上下文可以是作为提示的一部分提供的其他示例，以帮助模型更好地理解任务并引导预期的输出类型。

提示词通用技巧

从简单开始

在开始设计提示时，你应该记住，这实际上是一个迭代过程，需要大量的实验才能获得最佳结果。可以从简单的提示词开始，并逐渐添加更多元素和上下文（因为你想要更好的结果）。因此，在这个过程中不断迭代你的提示词是至关重要的。当你有一个涉及许多不同子任务的大任务时，可以尝试将任务分解为更简单的子任务，并随着结果的改善逐步构建。这避免了在提示设计过程中一开始就添加过多的复杂性。

指令

你可以使用命令来指示模型执行各种简单任务，例如“写入”、“分类”、“总结”、“翻译”、“排序”等，从而为各种简单任务设计有效的提示。请记住，你还需要进行大量实验以找出最有效的方法。以不同的关键词（keywords），上下文（contexts）和数据（data）试验不同的指令（instruction），看看什么样是最适合你特定用例和任务的。通常，上下文越具体、跟任务越相关则提示词的效果越好。

有些人建议将指令放在提示的开头。另有人则建议是使用像“###”这样的清晰分隔符来分隔指令和上下文。

例如：

```
### 指令 ###
将以下文本翻译成西班牙语：
文本：“hello! ”
```

输出：

```
¡Hola!
```

具体性

要非常具体地说明你希望模型执行的指令和任务。提示越具描述性和详细，结果越好。特别是当你对生成的结果或风格有要求时，这一点尤为重要。不存在什么特定的词元（tokens）或关键词（tokens）能确定带来更好的结果。更重要的是要有一个具有良好格式和描述性的提示词。事实上，在提示中提供示例对于获得特定格式的期望输出非常有效。在设计提示时，还应注意提示的长度，因为提示的长度是有限制的。想一想你需要多么的具体和详细。包含太多不必要的细节不一定是好的方法。这些细节应该是相关的，并有助于完成手头的任务。这是你需要进行大量实验的事情。

例如，让我们尝试从一段文本中提取特定信息的简单提示。

提示：

```
提取以下文本中的地名，所需格式为：
地点：<逗号分隔的公司名称列表>
输入：“虽然这些发展对研究人员来说是令人鼓舞的，但仍有许多谜团。里斯本未知的香帕利莫德中心的神经免疫学家
Henrique Veiga-Fernandes 说：“我们经常在大脑和我们在周围看到的效果之间有一个黑匣子。”“如果我们想在治疗
背景下使用它，我们实际上需要了解机制。””
```

输出：

```
地点：里斯本，香帕利莫德中心
```

避免不明确

给定上述关于详细描述和改进格式的建议，很容易陷入陷阱：想要在提示上过于聪明，从而可能创造出不明确的描述。通常来说，具体和直接会更好。这里的类比非常类似于有效沟通——越直接，信息传达得越有效。

例如，你可能有兴趣了解提示工程的概念。你可以尝试这样做：

```
解释提示工程的概念。保持解释简短，只有几句话，不要过于描述。
```

从上面的提示中不清楚要使用多少句子以及什么风格。尽管你可能仍会从上述提示中得到较好的响应，但更好的提示应当是非常具体、简洁并且切中要点的。例如：

使用 2-3 句话向学生解释提示工程的概念。

讲清楚做什么

设计提示时的另一个常见技巧是避免说不要做什么，而应该说要做什么。这样（说要做什么）更加的具体，并且聚焦于（有利于模型生成良好回复的）细节上。以下是一个电影推荐聊天机器人的示例，如果我们的指令关注于不要做什么，那么生成的内容很有可能并不是我们想要的。

提示：

以下是向客户推荐电影的代理程序。不要询问兴趣。不要询问个人信息。
客户：请根据我的兴趣推荐电影。
代理：

输出：

当然，我可以根据你的兴趣推荐电影。你想看什么类型的电影？你喜欢动作片、喜剧片、爱情片还是其他类型的电影？

以下是更好的提示：

提示：

以下是向客户推荐电影的代理程序。代理负责从全球热门电影中推荐电影。它应该避免询问用户的偏好并避免询问个人信息。如果代理没有电影推荐，它应该回答“抱歉，今天找不到电影推荐。”。
^^^ 顾客：请根据我的兴趣推荐一部电影。
客服：

输出：

抱歉，我没有关于你兴趣的任何信息。不过，这是目前全球热门的电影列表：[电影列表]。希望你能找到喜欢的电影！

提示词示例

文本概括

自然语言生成中的一个标准任务是文本摘要。文本摘要可以涵盖许多不同的风格和领域。事实上，语言模型最有前景的应用之一就是将文章和概念概括成简洁易读的摘要。让我们尝试使用提示进行一个基本的摘要任务。

提示词：

Explain antibiotics
A：

输出：

Antibiotics are a type of medication used to treat bacterial infections. They work by either killing the bacteria or preventing them from reproducing, allowing the body's immune system to fight off the infection. Antibiotics are usually taken orally in the form of pills, capsules, or liquid solutions, or sometimes administered intravenously. They are not effective against viral infections, and using them inappropriately can lead to antibiotic resistance.

"A:" 是一种在问答中使用的显式提示格式，你在这里使用它是为了告诉模型接下来你期望有一个答案。在这个例子中，我们可能还不清楚使用它是否有用，我们会在之后的示例中探讨这一点。现在假设我们感觉模型给了太多的信息，想要进一步提炼它。我们可以指导模型帮我们一句话总结相关内容：

提示词：

Antibiotics are a type of medication used to treat bacterial infections. They work by either killing the bacteria or preventing them from reproducing, allowing the body's immune system to fight off the infection. Antibiotics are usually taken orally in the form of pills, capsules, or liquid solutions, or sometimes administered intravenously. They are not effective against viral infections, and using them inappropriately can lead to antibiotic resistance.

Explain the above in one sentence:

译注： Explain the above in one sentence 可翻译为 用一句话解释上面的信息：

输出：

Antibiotics are medications used to treat bacterial infections by either killing the bacteria or stopping them from reproducing, but they are not effective against viruses and overuse can lead to antibiotic resistance.

信息提取

虽然语言模型是被训练用于执行自然语言生成及相关任务，但它同样非常擅长执行分类和一系列其他自然语言处理（NLP）任务。这是一个从给定段落中提取信息的提示词示例。

提示词：

Author-contribution statements and acknowledgements in research papers should state clearly and specifically whether, and to what extent, the authors used AI technologies such as ChatGPT in the preparation of their manuscript and analysis. They should also indicate which LLMs were used. This will alert editors and reviewers to scrutinize manuscripts more carefully for potential biases, inaccuracies and improper source crediting. Likewise, scientific journals should be transparent about their use of LLMs, for example when selecting submitted manuscripts.

Mention the large language model based product mentioned in the paragraph above:

输出：

The large language model based product mentioned in the paragraph above is ChatGPT.

问答

让模型给出具体回答的最佳方法之一是改善提示的格式。如前所述，一个提示可以结合指令、上下文、输入和输出指示（output indicators）来获得更好的结果。虽然这些组成部分不是必需的，但随着你给出的指令越具体，你将获得越好的结果，这成为了一种良好的实践。下面是一个例子，展示了按照更结构化的提示词来会是什么样子的。

提示词：

```
Answer the question based on the context below. Keep the answer short and concise.
Respond "Unsure about answer" if not sure about the answer.
Context: Teplizumab traces its roots to a New Jersey drug company called Ortho
Pharmaceutical. There, scientists generated an early version of the antibody, dubbed
OKT3. Originally sourced from mice, the molecule was able to bind to the surface of T
cells and limit their cell-killing potential. In 1986, it was approved to help prevent
organ rejection after kidney transplants, making it the first therapeutic antibody
allowed for human use.Question: What was OKT3 originally sourced from?
Answer:
```

输出：

```
Mice.
```

文本分类

到目前为止，你已经使用了简单的指令来执行任务。作为提示词工程师，你需要更擅长提供更好的指令。但这还不是全部！你还会发现，在更困难的用例中，仅提供指令是不够的。这就是你需要更多考虑上下文和其他元素（在提示词中使用）的地方。你可以提供的其他元素包括 输入数据 或 示例。

让我们通过一个文本分类的示例来演示这一点。

提示词：

```
Classify the text into neutral, negative or positive.
Text: I think the food was okay.
Sentiment:
```

输出：

```
Neutral
```

你给出了对文本进行分类的指令，模型以 'Neutral' 回应，这是正确的。这没有问题，但假设你真正需要的是让模型以你希望的确切格式给出标签。因此，你希望它返回的是 neutral 而不是 Neutral。你如何实现这一点呢？有几种不同的方法可以做到这一点。在这里，你关心的是具体性，所以你给提示词提供的信息越多，结果就越好。你可以尝试提供示例来指定正确的行为。让我们再试一次：

提示词：


```
Classify the text into neutral, negative or positive.  
Text: I think the vacation is okay.  
Sentiment: neutral  
Text: I think the food was okay.  
Sentiment:
```

输出:

```
neutral
```

完美！这次模型返回了 `neutral`，这是你所寻找的具体标签。看起来，提示中提供的示例帮助模型在其输出中保持了具体性。

对话

你可以通过提示工程进行更有趣的实验，比如指导大语言模型系统如何表现，指定它的行为意图和身份。当你在构建对话系统，如客户服务聊天机器人时，这尤其有用。比如，可以通过以下示例创建一个对话系统，该系统能够基于问题给出技术性和科学的回答。你可以关注我们是如何通过指令明确地告诉模型应该如何表现。这种应用场景有时也被称为 *角色提示 (Role Prompting)*。

提示词:

```
The following is a conversation with an AI research assistant. The assistant tone is  
technical and scientific.  
Human: Hello, who are you?  
AI: Greeting! I am an AI research assistant. How can I help you today?  
Human: Can you tell me about the creation of blackholes?  
AI:
```

输出:

```
Sure! Black holes are regions of spacetime where the gravitational force is so strong  
that nothing, not even light, can escape from it. They are created when a very massive  
star dies and its core collapses in on itself, forming a singularity of infinite density.  
The intense gravity of the singularity pulls in all the matter and radiation around it,  
creating the black hole.
```

我们的AI研究助手听起来有点太技术性了，对吗？好的，让我们改变这种行为，并指示系统给出更易于理解的回答。

提示词:

```
The following is a conversation with an AI research assistant. The assistant answers  
should be easy to understand even by primary school students.  
Human: Hello, who are you?  
AI: Greeting! I am an AI research assistant. How can I help you today?  
Human: Can you tell me about the creation of black holes?  
AI:
```

输出：

```
Sure! Black holes are created when a very massive star runs out of fuel and collapses in on itself. The gravity of the star is so strong that nothing, not even light, can escape its pull. This creates a point in space-time with an intense gravitational force called a black hole.
```

我想我们应该取得了一些进展，你也可以继续改进它。如果你再添加更多的示例，可能会得到更好的结果。

代码生成

大语言模型（LLMs）在代码生成方面非常有效。在此方面，Copilot 就是一个很好的示例。通过巧妙的提示，你可以执行大量的代码生成任务。下面我们来看几个示例。我们先用它写个简单的用户欢迎程序：

提示词：

```
/*Ask the user for their name and say "Hello"*/
```

输出：

```
let name = prompt("What is your name?");
console.log(`Hello, ${name}!`);
```

你看，我们甚至都不需要指定要使用的编程语言。来，我们再稍微升级一下。下面的示例展示了通过稍微多一点的提示词设计工作，LLM可以变得多么强大。

提示词：

```
"""
Table departments, columns = [DepartmentId, DepartmentName]
Table students, columns = [DepartmentId, StudentId, StudentName]
Create a MySQL query for all students in the Computer Science Department
"""
```

输出：

```
SELECT StudentId, StudentName
  FROM students
 WHERE DepartmentId IN (SELECT DepartmentId
                        FROM departments
                        WHERE DepartmentName = 'Computer Science');
```

这非常令人印象深刻。在这种情况下，你提供了关于数据库模式的数据，并要求它生成一个有效的 MySQL 查询。

推理

也许，当今大型语言模型（LLM）面临的最困难任务之一是需要某种形式的推理的任务。推理是最具吸引力的领域之一，因为有了推理，就可以从LLM中涌现出各种复杂的应用类型。目前，涉及数学能力的推理任务已经有了一些改进。对于当前的大型语言模型来说，执行推理任务可能会有一些难度，因此就需要更高级的提示词工程技术。我们会在后面的指南中介绍这些高级技术。现在，我们将介绍几个基本示例来展示算术功能。

提示词：

What is $9,000 \times 9,000$?

输出：

81,000,000

我们加大难度：

提示词：

The odd numbers in this group add up to an even number: 15, 32, 5, 13, 82, 7, 1.
A:

输出：

No, the odd numbers in this group add up to an odd number: 119.

这明显不对！我们试着用提示词去改进它：

提示词：

The odd numbers in this group add up to an even number: 15, 32, 5, 13, 82, 7, 1.
Solve by breaking the problem into steps. First, identify the odd numbers, add them, and indicate whether the result is odd or even.

输出：

Odd numbers: 15, 5, 13, 7, 1
Sum: 41
41 is an odd number.

效果好多了，但是顺便提一下，如果你多试几次，模型有时会给出错误的结果。如果你提供更好的指令并结合示例，这可能有助于获得更准确的结果。

提示技术

零样本提示

如今，经过大量数据训练并调整指令的LLM能够执行零样本任务。

提示：

将文本分类为中性、负面或正面。

文本：我认为这次假期还可以。

情感：

输出：

中性

请注意，在上面的提示中，我们没有向模型提供任何示例——这就是零样本能力的作用。当零样本不起作用时，建议在提示中提供演示或示例，这就引出了少样本提示。

少样本提示

Few-shot Prompts

- Few-shot prompting allows us to provide **exemplars** in prompts to steer the model towards better performance

The odd numbers in this group add up to an even number: 4, 8, 9, 15, 12, 2, 1.

A: The answer is False.

The odd numbers in this group add up to an even number: 17, 10, 19, 4, 8, 12, 24.

A: The answer is True.

The odd numbers in this group add up to an even number: 16, 11, 14, 4, 8, 13, 24.

A: The answer is True.

The odd numbers in this group add up to an even number: 17, 9, 10, 12, 13, 4, 2.

A: The answer is False.

The odd numbers in this group add up to an even number: 15, 32, 5, 13, 82, 7, 1.

A: The answer is True.

虽然大型语言模型展示了惊人的零样本能力，但在使用零样本设置时，它们在更复杂的任务上仍然表现不佳。少样本提示可以作为一种技术，以启用上下文学习，我们在提示中提供演示以引导模型实现更好的性能。演示作为后续示例的条件，我们希望模型生成响应。

提示：

“whatpu”是坦桑尼亚的一种小型毛茸茸的动物。一个使用whatpu这个词的句子的例子是：
我们在非洲旅行时看到了这些非常可爱的whatpus。
“farduddle”是指快速跳上跳下。
一个使用farduddle这个词的句子的例子是：

输出：

当我们赢得比赛时，我们都开始farduddle。

我们可以观察到，模型通过提供一个示例（即one-shot）已经学会了如何执行任务。对于更困难的任务，我们可以尝试增加演示（例如3个、5个、10个等）。标准的少样本提示对许多任务都有效，但仍然不是一种完美的技术，特别是在处理更复杂的推理任务时。让我们演示为什么会这样。您是否还记得之前提供的任务：

这组数字中的奇数加起来是一个偶数：15、32、5、13、82、7、1。
A：

如果我们再试一次，模型输出如下：

是的，这组数字中的奇数加起来是107，是一个偶数。

这不是正确的答案，这不仅突显了这些系统的局限性，而且需要更高级的提示工程。

让我们尝试添加一些示例，看看少样本提示是否可以改善结果。

提示：

这组数字中的奇数加起来是一个偶数：4、8、9、15、12、2、1。
A：答案是False。
这组数字中的奇数加起来是一个偶数：17、10、19、4、8、12、24。
A：答案是True。
这组数字中的奇数加起来是一个偶数：16、11、14、4、8、13、24。
A：答案是True。
这组数字中的奇数加起来是一个偶数：17、9、10、12、13、4、2。
A：答案是False。
这组数字中的奇数加起来是一个偶数：15、32、5、13、82、7、1。
A：

输出：

答案是True。

似乎少样本提示不足以获得这种类型的推理问题的可靠响应。上面的示例提供了任务的基本信息。如果您仔细观察，我们引入的任务类型涉及几个更多的推理步骤。换句话说，如果我们将问题分解成步骤并向模型演示，这可能会有所帮助。最近，思维链（CoT）已经流行起来，以解决更复杂的算术、常识和符号推理任务。总的来说，提供示例对解决某些任务很有用。当零样本提示和少样本提示不足时，这可能意味着模型学到的东西不足以在任务上表现良好。

COT提示

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Chain-of-Thought (CoT) Prompting

- Prompting can be further improved by instructing the model to reason about the task when responding
 - This is very useful for tasks that requiring reasoning
 - You can combine it with few-shot prompting to get better results
 - You can also do zero-shot CoT where exemplars are not available

The odd numbers in this group add up to an even number: 4, 8, 9, 15, 12, 2, 1.

A: Adding all the odd numbers (9, 15, 1) gives 25. The answer is False.

The odd numbers in this group add up to an even number: 15, 32, 5, 13, 82, 7, 1.

A: Adding all the odd numbers (15, 5, 13, 7, 1) gives 41. The answer is False.

思维链（CoT）提示通过中间推理步骤实现了复杂的推理能力，可以将其与少样本提示相结合，以获得更好的结果，以便在回答之前进行推理的更复杂的任务。

提示：

这组数中的奇数加起来是偶数：4、8、9、15、12、2、1。

A：将所有奇数相加（9、15、1）得到25。答案为False。

这组数中的奇数加起来是偶数：17、10、19、4、8、12、24。

A：将所有奇数相加（17、19）得到36。答案为True。

这组数中的奇数加起来是偶数：16、11、14、4、8、13、24。

A：将所有奇数相加（11、13）得到24。答案为True。

这组数中的奇数加起来是偶数：17、9、10、12、13、4、2。

A：将所有奇数相加（17、9、13）得到39。答案为False。

这组数中的奇数加起来是偶数：15、32、5、13、82、7、1。

A：

输出：

将所有奇数相加（15、5、13、7、1）得到41。答案为False。

还有一种方法是零样本CoT，它基本上涉及将“让我们逐步思考”添加到原始提示中。

Zero-Shot CoT

- Involves adding **"Let's think step by step"** to the original prompt

I went to the market and bought 10 apples. I gave 2 apples to the neighbor and 2 to the repairman. I then went and bought 5 more apples and ate 1. How many apples did I remain with?

11 apples

I went to the market and bought 10 apples. I gave 2 apples to the neighbor and 2 to the repairman. I then went and bought 5 more apples and ate 1. How many apples did I remain with?

Let's think step by step.

First, you started with 10 apples.
You gave away 2 apples to the neighbor and 2 to the repairman, so you had 6 apples left.
Then you bought 5 more apples, so now you had 11 apples.
Finally, you ate 1 apple, so you would remain with 10 apples.

让我们尝试一个简单的问题，看看模型的表现如何：

提示：

我去市场买了10个苹果。我给了邻居2个苹果和修理工2个苹果。然后我去买了5个苹果并吃了1个。我还剩下多少苹果？

输出：

11个苹果

答案是不正确的！现在让我们尝试使用特殊提示。

提示：

我去市场买了10个苹果。我给了邻居2个苹果和修理工2个苹果。然后我去买了5个苹果并吃了1个。我还剩下多少苹果？让我们逐步思考。

输出：

首先，您从10个苹果开始。
您给了邻居和修理工各2个苹果，所以您还剩下6个苹果。
然后您买了5个苹果，所以现在您有11个苹果。
最后，您吃了1个苹果，所以您还剩下10个苹果。

令人印象深刻的是，这个简单的提示在这个任务中非常有效。这在您没有太多示例可用于提示时特别有用。

自我一致性

自我一致性旨在“替换链式思维提示中使用的天真贪婪解码方法”。其想法是通过少样本 CoT 采样多个不同的推理路径，并使用生成结果选择最一致的答案。这有助于提高 CoT 提示在涉及算术和常识推理的任务中的性能。

让我们尝试以下算术推理示例：

提示：

当我6岁时，我的妹妹是我的一半年龄。现在我70岁了，我的妹妹多大？

输出：

35

输出是错误的！我们如何通过自我一致性来改进这个问题？

提示：

Q：林中有15棵树。林业工人今天将在林中种树。完成后，将有21棵树。林业工人今天种了多少棵树？
A：我们从15棵树开始。后来我们有21棵树。差异必须是他们种树的数量。因此，他们必须种了 $21 - 15 = 6$ 棵树。答案是6。
Q：停车场有3辆汽车，又来了2辆汽车，停车场有多少辆汽车？
A：停车场已经有3辆汽车。又来了2辆。现在有 $3 + 2 = 5$ 辆汽车。答案是5。
Q：Leah有32块巧克力，她的姐姐有42块。如果他们吃了35块，他们总共还剩多少块？
A：Leah有32块巧克力，Leah的姐姐有42块。这意味着最初有 $32 + 42 = 74$ 块巧克力。已经吃了35块。因此，他们总共还剩 $74 - 35 = 39$ 块巧克力。答案是39。
Q：Jason有20个棒棒糖。他给Denny一些棒棒糖。现在Jason只有12个棒棒糖。Jason给Denny多少棒棒糖？

A: Jason有20个棒棒糖。因为他现在只有12个,所以他必须把剩下的给Denny。他给Denny的棒棒糖数量必须是 $20-12=8$ 个棒棒糖。答案是8。

Q: Shawn有五个玩具。圣诞节,他从他的父母那里得到了两个玩具。他现在有多少个玩具?

A: 他有5个玩具。他从妈妈那里得到了2个,所以在那之后他有 $5+2=7$ 个玩具。然后他从爸爸那里得到了2个,所以总共他有 $7+2=9$ 个玩具。答案是9。

Q: 服务器房间里有9台计算机。从周一到周四,每天都会安装5台计算机。现在服务器房间里有多少台计算机?

A: 从周一到周四有4天。每天都添加了5台计算机。这意味着总共添加了 $4*5=20$ 台计算机。一开始有9台计算机,所以现在有 $9+20=29$ 台计算机。答案是29。

Q: Michael有58个高尔夫球。星期二,他丢失了23个高尔夫球。星期三,他又丢失了2个。星期三结束时他还剩多少个高尔夫球?

A: Michael最初有58个球。星期二他丢失了23个,所以在那之后他有 $58-23=35$ 个球。星期三他又丢失了2个,所以现在他有 $35-2=33$ 个球。答案是33。

Q: Olivia有23美元。她用每个3美元的价格买了五个百吉饼。她还剩多少钱?

A: 她用每个3美元的价格买了5个百吉饼。这意味着她花了15美元。她还剩8美元。

Q: 当我6岁时,我的妹妹是我的一半年龄。现在我70岁了,我的妹妹多大?

A:

输出 1:

当我6岁时,我的妹妹是我的一半年龄,也就是3岁。现在我70岁了,所以她是 $70-3=67$ 岁。答案是67。

输出 2:

当叙述者6岁时,他的妹妹是他年龄的一半,也就是3岁。现在叙述者70岁了,他的妹妹应该是 $70-3=67$ 岁。答案是67。

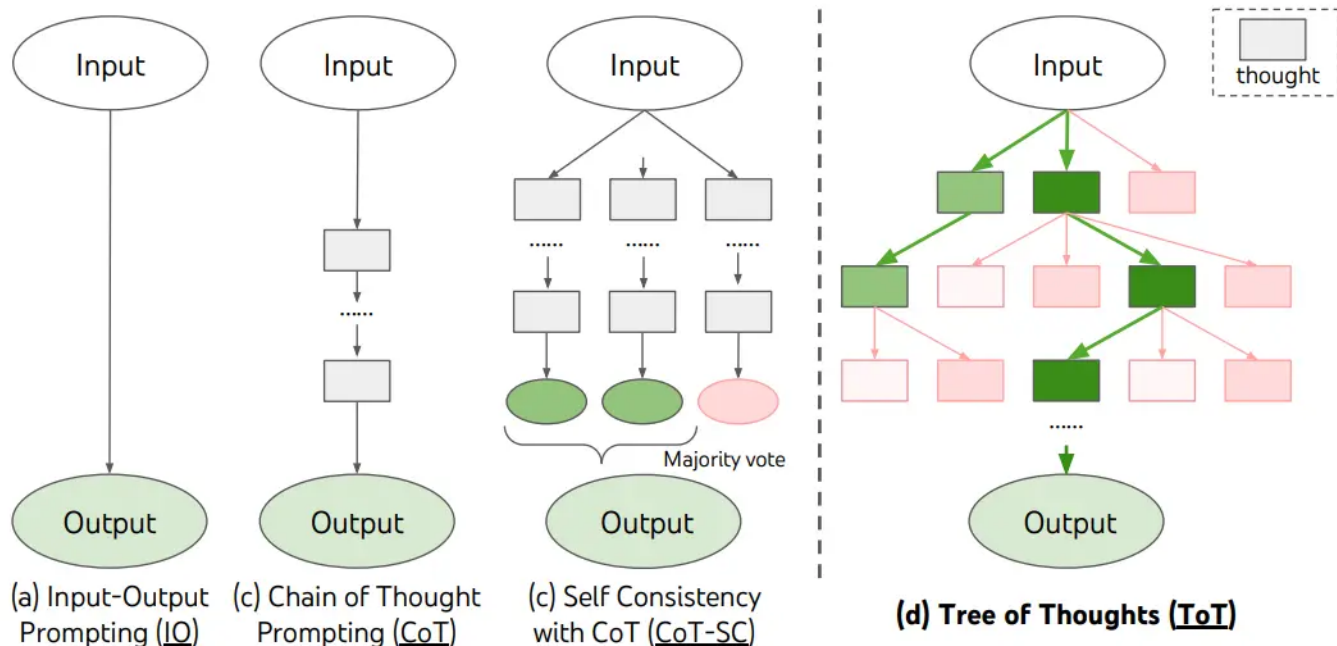
输出 3:

当我6岁时,我的妹妹是我的一半年龄,也就是3岁。现在我70岁了,所以她是 $70/2=35$ 岁。答案是35。

思维树 (ToT)

对于需要探索或预判战略的复杂任务来说,传统或简单的提示技巧是不够的。有人提出了思维树(ToT)框架,该框架基于思维链提示进行了总结,引导语言模型探索把思维作为中间步骤来解决通用问题。ToT 维护着一棵思维树,思维由连贯的语言序列表示,这个序列就是解决问题的中间步骤。使用这种方法,LLM 能够自己对严谨推理过程的中间思维进行评估。LLM 将生成及评估思维的能力与搜索算法(如广度优先搜索和深度优先搜索)相结合,在系统性探索思维的时候可以向前验证和回溯。

ToT 框架原理如下:

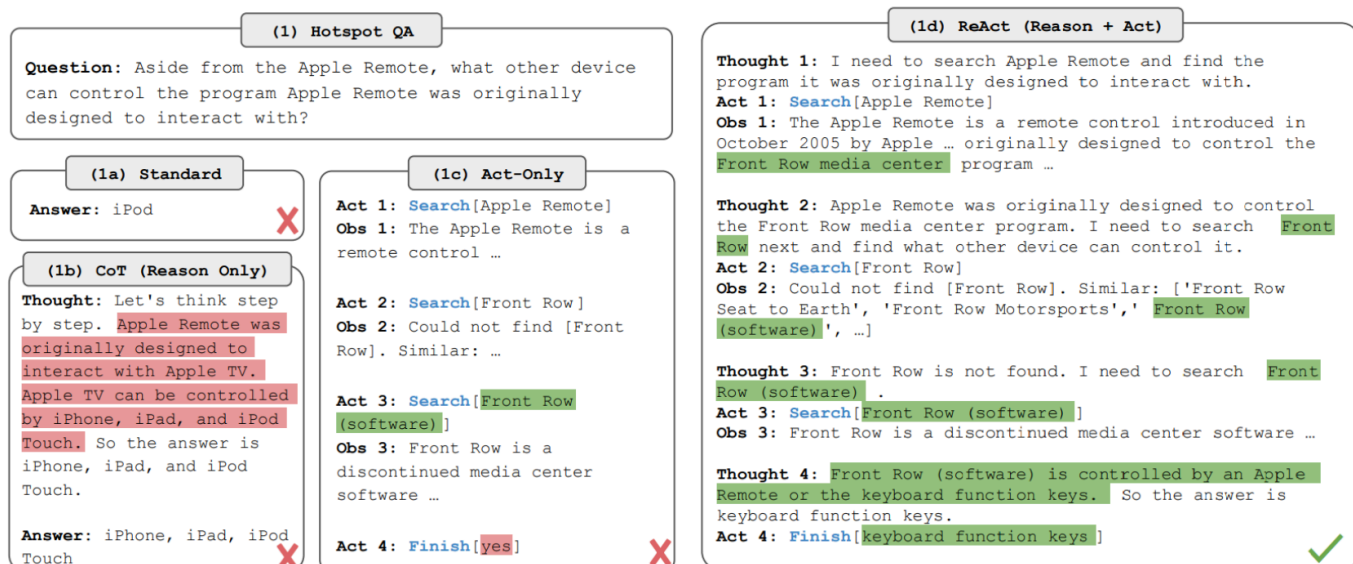


检索增强生成（RAG）

通用语言模型通过微调就可以完成几类常见任务，比如分析情绪和识别命名实体。这些任务不需要额外的背景知识就可以完成。要完成更复杂和知识密集型的任务，可以基于语言模型构建一个系统，访问外部知识源来做到。这样的实现与事实更加一致性，生成的答案更可靠，还有助于缓解“幻觉”问题。

Meta AI 的研究人员引入了一种叫做检索增强生成（Retrieval Augmented Generation）的方法来完成这类知识密集型的任务。RAG 把一个信息检索组件和文本生成模型结合在一起。RAG 可以微调，其内部知识的修改方式很高效，不需要对整个模型进行重新训练。RAG 会接受输入并检索出一组相关/支撑的文档，并给出文档的来源（例如维基百科）。这些文档作为上下文和输入的原始提示词组合，送给文本生成器得到最终的输出。这样 RAG 更加适应事实会随时间变化的情况。这非常有用，因为 LLM 的参数化知识是静态的。RAG 让语言模型不用重新训练就能够获取最新的信息，基于检索生成产生可靠的输出。这说明 RAG 是一种可行的方案，能在知识密集型任务中增强语言模型的输出。最近，基于检索器的方法越来越流行，经常与 ChatGPT 等流行 LLM 结合使用来提高其能力和事实一致性。

ReAct框架



```
llm = OpenAI(model_name="text-davinci-003", temperature=0)
tools = load_tools(["google-serper", "llm-math"], llm=llm)
agent = initialize_agent(tools, llm, agent="zero-shot-react-description", verbose=True)
agent.run("奥利维亚·王尔德的男朋友是谁?他现在的年龄的0.23次方是多少?")
```

链执行如下所示:

```
> 正在输入新代理执行器链.....
    我得查出奥利维亚·王尔德的男友是谁然后计算出他的年龄的 0.23 次方。
操作: 搜索
操作输入: "奥利维亚·王尔德的男友"
观察: 奥利维亚·王尔德与杰森·苏代基斯在多年前订婚, 在他们分手后, 她开始与哈里·斯泰尔斯约会 — 参照他们的关系时间线。
思考: 我需要找出哈里·斯泰尔斯的年龄。
操作: 搜索
操作输入: "哈里·斯泰尔斯的年龄"
观察: 29 岁
思考: 我需要计算 29 的 0.23 次方。
操作: 计算器
操作输入: 29^0.23
观察: 答案: 2.169459462491557

思考: 现在我知道最终答案了。
最终答案: 哈里·斯泰尔斯, 奥利维亚·王尔德的男朋友, 29 岁。他年龄的 0.23 次方是 2.169459462491557。

> 结束链。
```

我们得到如下输出:

```
"哈里·斯泰尔斯, 奥利维亚·王尔德的男朋友, 29 岁。他年龄的 0.23 次方是 2.169459462491557。"
```